Descripción general: Sistema de citas médicas EPS Citas

El sistema **EPS Citas** (Asosalud IPS) es una plataforma web integral para la gestión de citas médicas. Facilita la interacción entre pacientes, profesionales médicos y administradores dentro de una organización de atención médica integrada (IPS). La aplicación automatiza el ciclo de vida de una cita médica, desde el autorregistro y la programación de la cita por parte del paciente hasta las notas de la consulta clínica y la supervisión administrativa.

El sistema está construido sobre una pila moderna de Python que utiliza el microframework **Flask , con especial atención al control de acceso basado en roles (RBAC) y la integridad de los datos a través de una base de datos relacional MySQL** .

Arquitectura del sistema

La aplicación sigue un patrón modular inspirado en "App Factory", donde la lógica central se separa en módulos de planos, modelos y utilidades.

Conjunto de tecnologías

- **Backend:** Python 3.11+ usando Flask 3.0LÉAME.md13-91
- **Base de datos:** MySQL 8.0+ usando**mysql-connector-python** base de datos.py1-3 LÉAME.md14-92
- **Seguridad: bcrypt** para el hash de contraseñas y decoradores personalizados para RBAC basado en sesiones.modelos/usuarios.py1-5 utils/decoradores.py1-10
- **Interfaz de usuario:** plantillas Jinja2, CSS3 con variables personalizadas y JavaScript puro para la validación del lado del cliente.aplicación.py40-130 static/css/style.css1-50 static/js/main.js1-20

Estructura del proyecto

```
eps_citas_app/
├── app.py                  ← Punto de entrada Flask (rutas)
├── config.py               ← Configuración (lee el .env)
├── database.py             ← Gestión de conexión MySQL
├── requirements.txt        ← Dependencias Python
├── .env                    ← Variables de entorno (NO subir a Git)
├── .env.example            ← Plantilla del .env (sí subir a Git)
├── .gitignore
├── eps_citas_db.sql        ← Script para crear la base de datos
├── models/
│   ├── __init__.py
│   ├── pacientes.py        ← CRUD pacientes + validaciones
│   └── citas.py            ← CRUD citas + validaciones
├── templates/
│   ├── base.html           ← Layout base (navbar, footer)
│   ├── index.html          ← Página principal
│   ├── registro_paciente.html
│   ├── reservar_cita.html
│   ├── consulta_cita.html
│   ├── resultado_cita.html
│   └── error.html
└── static/
    ├── css/style.css       ← Estilos propios (diseño médico)
    └── js/main.js          ← Validaciones y UI
```

Roles y permisos de usuario

El sistema impone un estricto control de acceso a través del **rol_required**decorador.utils/decoradores.py28-45Las identidades de usuario se dividen en una **usuarios**tabla base y tablas de perfil específicas (**pacientes**, **medico**

Roles	Responsabilidades principales	Nivel de acceso
Paciente	Autorregistro, reserva de citas, consulta del historial personal.	Acceso a**routes/paciente.py**
Médico	Gestionar la agenda diaria, asistir a citas, redactar notas clínicas.	Acceso a**routes/medico.py**
Administración	Configuración del sistema, gestión del personal médico, activación/desactivación de usuarios.	Acceso a**routes/admin.py**

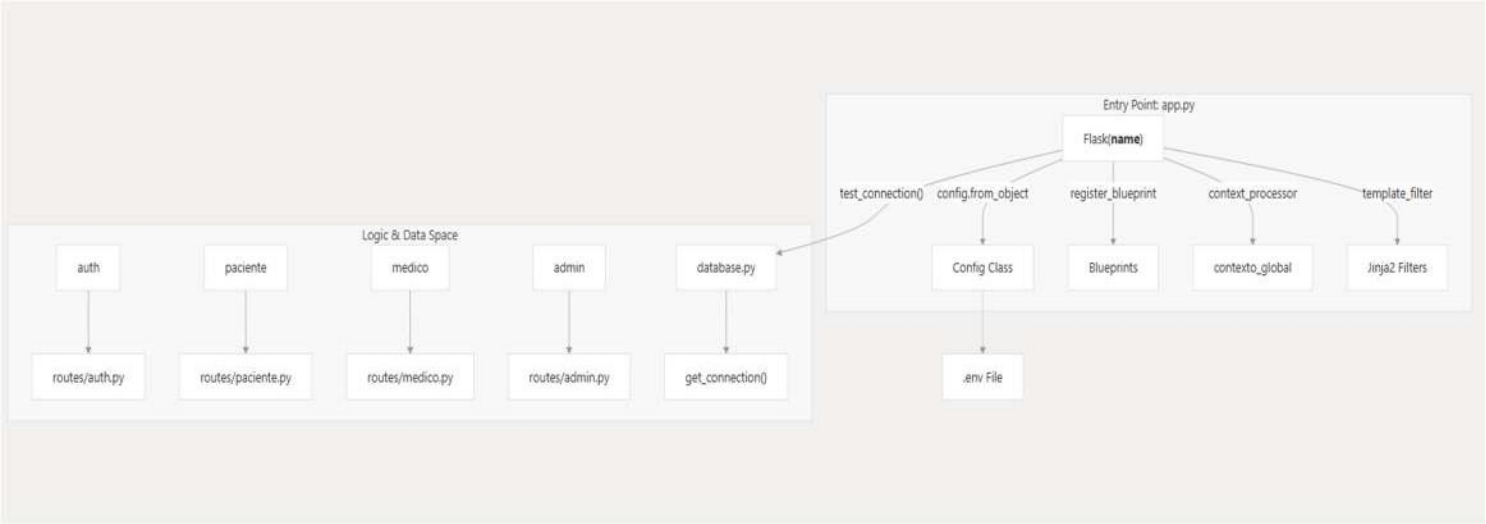
Arquitectura de la aplicación

La aplicación EPS Citas está construida con el framework web Flask, siguiendo un patrón **Application Factory para la inicialización y una arquitectura modular Blueprint** para el enrutamiento. El sistema separa las responsabilidades en una capa de conectividad de base de datos, modelos de datos para la lógica de negocio y una capa de presentación basada en plantillas Jinja2.

Fábrica de aplicaciones e inicialización

El punto de entrada de la aplicación es **app.py**, que inicializa la instancia de Flask y aplica las configuraciones del **Config**objetoaplicación.py11-12Durante el inicio, la aplicación realiza una comprobación de conectividad con la base de datos MySQL utilizando**test_connection()** aplicación.py36

El siguiente diagrama ilustra el flujo de inicialización de alto nivel y cómo se relacionan las entidades de código con los componentes estructurales del sistema:



Tecnologías Principales

Backend
app.py:1-12
MySQL 8.0+ como base de datos relacional, usando `mysql-connector-pytmysql-connector-python config.py:10-32
bcrypt para hash seguro de contraseñas

Interfaz
Jinja2para plantillas del lado del servidor
CSS3 con variables personalizadas para temas por rol
JavaScript vanilla para validación de cliente

Seguridad y Autenticación
Sesiones basadas en Flask con duración de 8 horasconfig.py:38-41
Decoradores personalizados para RBACrol_requiredadmin.py:14
Control de acceso por roles: paciente, médico, admin

Servicios y Arquitectura
Patrón App Factory
app.py:5-19

- **routes/auth**- Autenticación
- **routes/paciente**- Funcionalidades paciente
- **routes/medico**- Funcionalidades médico
- **routes/admin**- Administración del sistema

Gestión de Base de Datos

- Conexión directa a MySQL mediante**database.py**
- config.py:11-32

Despliegue

- **Ferrocarril** como plataforma de despliegue en producción
- Variables de entorno inyectadas automáticamente por Railway
- **.env**config.py:5-6

Modelos de datos

El **models/**paquete contiene la lógica de negocio principal y la capa de acceso a datos para el sistema EPS Citas. La aplicación utiliza un enfoque tradicional Modelo-Vista-Controlador (MVC) donde los modelos se implementan como clases de Python con métodos estáticos que interactúan directamente con la base de datos MySQL a través de la **database.py**utilidad

Modelo de usuario: autenticación y gestión de cuentas

La **Usuario**clase **models/usuarios.py**maneja el perímetro de seguridad de la aplicación. Gestiona el hash de contraseñas utilizando **bcrypt**con un factor de costo de 12.modelos/usuarios.py14-15y proporciona métodos de fábrica para crear diferentes tipos de usuarios.

- **Autenticación:** El **login**método verifica las credenciales y comprueba si el **activo**indicador está configurado en 1.modelos/usuarios.py44-69
- **Registro:** Proporciona**registrar_paciente** modelos/usuarios.py81y**crear_medico** modelos/usuarios.py113que gestionan la inserción inicial **usuarios**de la tabla antes de la creación del perfil.
- **Política de seguridad:** Aplica complejidad de contraseña (mínimo 8 caracteres, 1 mayúscula, 1 dígito) mediante**validar_password**

Modelos Paciente y Médico — Gestión del Perfil

Estos modelos gestionan los metadatos asociados a las entidades humanas en el sistema. Siguen un patrón de creación en dos pasos: **Usuario**primero se crea un modelo y su metadato **lastrowid**se pasa al **crear**método del modelo de perfil.modelos/pacientes.py36-40 modelos/médicos.py37-38

- **Paciente:** Almacena el **eps**campo y proporciona métodos para recuperar perfiles por **documento**o**usuario_id** modelos/pacientes.py71-101
- **Médico:** Gestiona el **ESPECIALIDADES**enummodelos/médicos.py4y proporciona consultas avanzadas para citas, incluyendo **citas_hoy**el **estadisticas**panel de control del médico.

Modelo Cita: Ciclo de vida de la cita

La **Cita**clase **models/citas.py**controla la funcionalidad principal de la aplicación: la programación de citas médicas. Actúa como un puente entre pacientes y médicos.

- **Gestión estatal:** Los nombramientos se mueven a través de los estados definidos en **ESTADOS_CITA**: **Pendiente**, **Atendida**, **Cancelada**, o**No asistió** modelos/citas.py6
- **Reglas de negocio:** El modelo impone compatibilidad entre **tipo_cita**y el médico **especialidad**usando el **ESPECIALIDAD_TIPOS**mapamodelos/citas.py9-13
- **Acceso a datos:** Utiliza uniones complejas y la **v_citas_completas**vista de la base de datos para proporcionar información detallada sobre las citas para las vistas administrativas y clínicas.modelos/citas.py214-230

Para obtener más detalles, consulte el modelo Cita: ciclo de vida de la cita .

Rutas y planos

La aplicación EPS Citas está estructurada mediante **Flask Blueprints** para separar las responsabilidades según los roles de usuario y la funcionalidad del sistema. Este enfoque modular permite que la aplicación sea escalable al aislar la lógica de autenticación, los servicios al paciente, la gestión médica y la supervisión administrativa en módulos distintos.

Todos los planos se importan y registran en el punto de entrada principal.**app.py** aplicación.py5-19

Registro y enrutamiento de planos

La aplicación define cuatro planos principales, cada uno asociado a un prefijo de URL específico y un conjunto de responsabilidades. Cuando un usuario accede a la URL raíz **/**, la aplicación evalúa su sesión y lo redirige al panel de control apropiado según su rol.aplicación.py23-33

Cianotipo	Módulo	Prefijo de URL	Responsabilidad principal
auth	**routes.auth**	**/auth**	Autenticación, cierre de sesión y registro de pacientes.
paciente	**routes.paciente**	**/paciente**	Reserva de citas, historial y gestión de perfiles.
medico	**routes.medico**	**/medico**	Consulta con el paciente, notas clínicas y gestión de la agenda.
admin	**routes.admin**	**/admin**	Gestión de usuarios, creación de médicos y métricas del sistema.

Descripción general del enrutamiento del sistema

Este diagrama ilustra cómo el **app.py**punto de entrada dirige las solicitudes entrantes a los planos específicos en función de la ruta URL y el estado de la sesión.

Fuentes:aplicación.py15-33 utils/decoradores.py16-35

Servicios públicos compartidos y decoradores

Para garantizar la seguridad y la coherencia en todas las rutas, la aplicación utiliza un conjunto de decoradores y asistentes de sesión definidos en **utils/decoradores.py**. Estas utilidades gestionan el "Espacio de Entidades de Código" de la sesión del usuario y aplican el Control de Acceso Basado en Roles (RBAC).

Decoradores de seguridad

- **@login_required**: Envuelve funciones para asegurar que **usuario_id**exista en la sesión. Si no, redirige a**auth.login** utils/decoradores.py5-13
- **@rol_required(*roles)**: Amplía la protección de inicio de sesión verificando que **session.get("rol")**coincida con uno de los roles autorizados para ese punto final específico.utils/decoradores.py16-35

Ayudantes para la gestión de sesiones

La aplicación centraliza la manipulación de la sesión para evitar **session**la modificación directa de objetos en múltiples archivos:

- **set_sesion(usuario, nombre, perfil_id)**Rellena la sesión de Flask con la identidad del usuario y los datos de perfil después de una autenticación exitosa.utils/decoradores.py57-65
- **clear_sesion()**: Se invoca **session.clear()**para cerrar la sesión de un usuario de forma segura.utils/decoradores.py67-70
- **get_usuario_sesion()**: Devuelve un diccionario con los datos de la sesión actual para su uso en la lógica o en la representación de plantillas.utils/decoradores.py46-54

Mapeo de datos de sesión

Este diagrama establece un vínculo entre el concepto de "Sesión de usuario" y las claves específicas utilizadas en el **Flask.session**objeto y la **set_sesion**utilidad.

Fuentes:utils/decoradores.py46-70

Documentación detallada del plano

Para obtener información detallada sobre las rutas, la lógica y las plantillas asociadas a cada plano, consulte las siguientes páginas secundarias:

Plan de autenticación: inicio de sesión, cierre de sesión y registro.

Gestiona los puntos de entrada y salida de la aplicación. Incluye el flujo de inicio de sesión, la inicialización de la sesión **set_sesion**y el proceso de autorregistro para nuevos pacientes.

- **Archivo clave: routes/auth.py**

Plan de Pacientes — Portal del Paciente

Contiene opciones para que los pacientes vean su panel de control, gestionen sus citas (reservas, edición y cancelaciones) y actualicen su información personal del EPS.

- **Archivo clave: routes/paciente.py**

Plan de Medicina — Portal del Médico

Se centra en la agenda médica. Incluye el panel de control del médico con estadísticas diarias, la posibilidad de ver el historial detallado del paciente y el flujo de trabajo para añadir notas clínicas a las citas.

- **Archivo clave: routes/medico.py**

Plan de administración — Portal administrativo

Proporciona una supervisión general del sistema. Incluye la gestión de cuentas de usuario (activación/desactivación de usuarios), la creación de perfiles de médicos y la agregación de métricas de todo el sistema.

- **Archivo clave: routes/admin.py**

Seguridad y gestión de sesiones

Esta página describe la arquitectura de seguridad de la aplicación EPS Citas, centrándose en cómo se protege la identidad del usuario, cómo se restringe el acceso mediante decoradores y el ciclo de vida de una sesión de usuario. El sistema emplea prácticas estándar del sector para el cifrado de contraseñas y la persistencia de sesión, a fin de garantizar la integridad y la confidencialidad de los datos.

Autenticación y seguridad de contraseñas

La aplicación utiliza un modelo de autenticación robusto centrado en la **Usuario**clasemodelos/usuarios.py6-7La seguridad de las contraseñas se gestiona mediante la **bcrypt**biblioteca.requisitos.txt4que implementa un algoritmo de hash con sal para protegerse contra ataques de tabla arcoíris y de fuerza bruta.

- **Hashing** : Las contraseñas nunca se almacenan en texto plano. Se procesan utilizando **bcrypt.hashpw**un factor de coste de **12.**modelos/usuarios.py14-15

- **Verificación** : Durante el inicio de sesión, la contraseña proporcionada se verifica con el hash almacenado utilizando**bcrypt.checkpw** modelos/usuarios.py18-22
- **Estado de la cuenta** : El sistema incluye un **activo**indicador. Si un administrador desactiva a un usuario, este no podrá iniciar sesión aunque tenga las credenciales correctas.modelos/usuarios.py63-64

Para obtener información detallada sobre el proceso de verificación de credenciales y las reglas de complejidad de las contraseñas, consulte la sección "Política de contraseñas y flujo de autenticación" .

Mapeo del modelo de seguridad al código

El siguiente diagrama ilustra cómo se asignan los conceptos de seguridad a funciones y variables específicas dentro del código fuente.

Lógica del flujo de autenticación

Fuentes:modelos/usuarios.py44-74 utils/decoradores.py57-64

Gestión de sesiones

La aplicación gestiona el estado del usuario a través del objeto de sesión de Flask, configurado para garantizar la durabilidad y la seguridad en la configuración de la aplicación.

Característica	Configuración / Implementación	Referencia de archivo
Llave secreta	Cargado desde **SECRET_KEY**la variable de entorno para la firma de cookies	config.py35
Tipo de sesión	Permanente (se mantiene incluso después de reiniciar el navegador)	config.py39
Vida	8 horas (**PERMANENT_SESSION_LIFETIME**)	config.py40
Limpieza	**session.clear()**al cerrar sesión	utils/decoradores.py67-70

Tras iniciar sesión correctamente, la **set_sesion**utilidad rellena la sesión con el ID del usuario, el documento, el rol, el nombre para mostrar y su ID de perfil específico (ya sea de la tabla **pacientes**o).**medicos**utils/decoradores.py57-64

Control de acceso

El acceso a rutas específicas se controla mediante decoradores de Python ubicados en **utils/decoradores.py**. Estos decoradores actúan como middleware, interceptando las solicitudes antes de que lleguen a la lógica del controlador.

- **@login_required**: Asegura que **usuario_id**exista una sesión. Si no, el usuario es redirigido a la página de inicio de sesión.utils/decoradores.py5-13
- **@rol_required(*roles)**: Un decorador de orden superior que comprueba si el rol del usuario (almacenado en la sesión) coincide con los roles permitidos para la ruta.utils/decoradores.py16-35

Aplicación del control de acceso

Fuentes:utils/decoradores.py16-44

Para obtener información detallada sobre cómo se definen y aplican los roles (paciente, médico, administrador) en la interfaz de usuario y el sistema, consulte Control de acceso basado en roles (RBAC) .

---

protecciones administrativas

El sistema incluye salvaguardias específicas para las acciones administrativas:

- **Suspensión de cuenta** : Los administradores pueden cambiar el **activo**estado de cualquier usuario a través del **toggle_activo**métodomodelos/usuarios.py236-258
- **Autoprotección** : El sistema impide que los administradores desactiven sus propias cuentas para evitar bloqueos accidentales.modelos/usuarios.py240-242

Fuentes:modelos/usuarios.py236-258 config.py9-45 utils/decoradores.py1-70